# Non-intrusive Cache Side Channel Protection Near Memory

MICRO 2026 Submission **#NaN** – Confidential Draft – Do NOT Distribute!!

## Abstract

Historically, identical code or data share the same memory pages. But it was later discovered that memory sharing enables a reuse-based side channel that can leak non-shared data. This causes data centers to disable memory sharing across security domains (e.g., different VMs, containers), unfortunately contradicting a main goal of cloud computing – boosting cost efficiency via resource sharing.

Prior work proposes hardware defenses to securely restore memory sharing. However, these approaches require modifications across numerous components of the CPU—including L1 and L2 caches, the LLC, coherence directory, MMU, and memory controller. Such widespread changes are highly intrusive and can significantly delay, and thus far have prevented, real-world use.

We propose a non-intrusive design that enables global (i.e., all caches + memory) protection against reuse-based side channels via only localized modifications (i.e., by placing all hardware protection logic within the memory controller) and incorporate the protection into a high-end server. Real-system evaluations show secure ***elimination*** of cache reuse side channels across containers, VMs, and processes, despite only modifying the memory controller.

## Keywords

Memory Sharing, Side Channel, Caches, Near Memory Computing

## 1 Introduction

Significant memory contents are often identical. For example, different VMs and containers can use the same ***code***, such as C, Python, and cryptography libraries, OS kernels, and utilities. As another example, cloud-based services can dynamically launch more VM/-container copies of themselves to scale throughput according to the incoming demand elastically; multiple VMs/containers of the same user can have large volumes of identical memory ***data***.

Historically, identical contents share the same memory pages. This avoids wasting memory on redundant data copies. As memory becomes the dominant cost factor for data centers – responsible for 40% to 50% of total infrastructure costs and rising [42] – wasting memory is costly. A core goal of cloud computing – improving cost efficiency via resource sharing – also calls for memory sharing.

Unfortunately, it was later discovered that memory sharing across security domains (e.g., different VMs, containers) opens up a **reuse-based side channel** that leaks non-shared information when shared memory blocks recently accessed by one domain are reused (i.e., re-accessed) by another (e.g., a malicious) domain.

***In caches***, victim accesses to memory blocks shared with the attacker can cause attacker-observable latency differences [19, 20, 28, 29, 40, 45, 78–80, 82]. For example, the Flush+Reload attack can detect victim accesses to a shared block when observing cache hit latency on first access to the block after flushing the block; if the victim's access depends on a secret, the secret will be inferred [78].

***In memory***, after a victim accesses a shared location, an attacker can observe a row buffer hit (instead of a miss) when accessing the same location, creating a reuse-based memory side channel [56].

Together, these reuse-based side channels pose a severe vulnerability. As a result, cloud providers disabled memory sharing across security domains (e.g., VMs, containers) [7–9][48][27], even across different VMs of the same user [48]; memory sharing across the same user's VMs would allow malware that has entered one of the user's VMs to launch reuse attacks on the user's other VMs.

To securely restore memory sharing, prior works [23, 49, 51, 60, 66, 76, 77] propose various hardware protections against both cache reuse and memory reuse side channels. Of the two, cache protection is more complex due to the multi-level and distributed organization of modern cache hierarchy, which comprises a tiled LLC with a separate coherence directory and numerous per-core L1/L2 caches; as a result, prior cache protection works modify L3, L2, and L1 caches, the coherence directory, and the MMU (i.e., to implement the new hardware interfaces for the OS to use the new cache features). Thus, achieving global (i.e., both cache and memory) protection by combining prior hardware defenses requires ***global*** modifications in the CPUs ***(i.e., effectively modifying every CPU component except for the core pipeline, see Table 1)***.

| Components to Change for Global Protection | MC | LLC | L2 Caches | L1 Caches | Cache Directory | MMU (for new SW/HW interface) | OS |
|---|---|---|---|---|---|---|---|
| When combining Prior Cache Protection with Prior Memory Protection | ▲ | ▲ | ▲ | ▲ | ▲ | ▲ | ▲ |
| Cache Protection Near Memory (this work) + Prior Memory Protection | ▲ | — | — | — | — | — | ▲ |

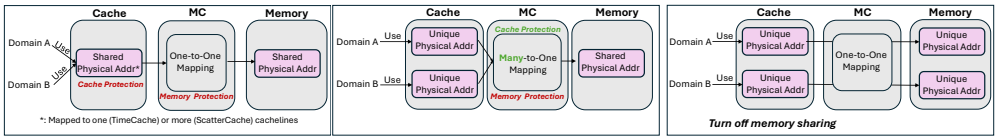
**Table 1: Components to modify to provide global protection.**

Making changes globally across the CPU is intrusive and can delay (and perhaps prevent) real-world use. There are myriads of different side channels [11]; making global CPU modifications per side channel is impractical. Instead, side channel protections that see real-world use are all localized to a single CPU component; for example, SSBD [31] is within the core, while CAT [32] (a performance feature that could be reused for security) is localized to the LLC (i.e., to the LLC's replacement policy and its MSR).

**In this paper,** we explore a non-intrusive design where making ***local changes*** to one hardware component can achieve ***global protection*** against cross-domain reuse side channels. Specifically, we propose moving cache protection ***near memory*** (i.e., into the memory controller, a.k.a MC) to avoid all modifications to the cache hierarchy. We call our cache protection placed remotely near memory *Non-intrusive **C**ache Side Channel **P**rotection Near **M**emory (CPM).*

To move cache protection near memory, we ***observe*** caches face the reuse side channel due to containing shared physical addresses. In conventional systems and prior works, caches contain shared physical addresses because unique addresses are merged into shared addresses before entering caches; the OS shares the same physical

*MICRO 2026, Athens, Greece*
2026. ACM ISBN 978-X-XXXX-XXXX-X/XX/XX
https://doi.org/XXXXXXX.XXXXXXX




(a) Prior hardware defenses. (b) CPM (our work). (c) Current system deployment.

Figure 1: Key architectural differences.

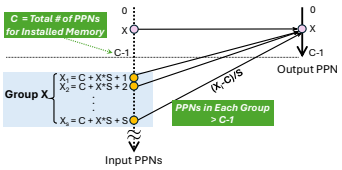



Figure 2: CPM uses Table-free merging.

page across virtual pages with identical values by recording the same physical page number (PPN) in the page table entries, causing the MMU to send shared physical addresses to caches.

**High-level Idea:** We propose delaying the merging of different addresses into shared addresses after the requests leave the cache hierarchy – when they arrive at the MC. Specifically, CPM enhances the MC to merge unique PPNs to the same/shared PPN (see Figure 1b). CPM guarantees elimination of reuse-based cache side channels across security domains and, thus, achieves identical cache security to mainstream data center deployments, which disable sharing cross domains. In data centers today, different security domains use different cachelines (i.e., they keep many copies of identical memory values redundantly in caches, see Figure 1c) due to disabling memory sharing; having independent copies in caches for different security domains guarantees no cacheline reuse across domains and, thus, no cache reuse side channels. By delaying the merging of addresses until the MC, CPM also disables sharing in caches (see Figure 1b), *while still sharing in memory* (see Figure 1b).

**Design challenge and Solution**: A practical protection should also incur minimal performance overhead. As such, a key challenge is how to minimize performance overheads while merging physical addresses at the MC. While executing memory instructions, existing systems merge virtual addresses to shared physical addresses by relying on the MMU to look up page tables, which record the shared PPNs. Reusing this table-based approach for address merging at the MC requires adding a new (i.e., PPN-to-PPN) mapping table to the MC and incurring new latency and bandwidth overheads to perform additional address translation using the new table.

We propose a novel table-free merging that enhances the MC to merge each fixed[1] group of PPNs to the same PPN (i.e., map all PPNs in group $x$ to PPN $x$, see Figure 2); since the group of PPNs that merge to the same PPN is fixed, the output PPN for a given input PPN can be computed directly using a predefined mathematical formula (see Figure 2), without any table access; to use CPM to share an arbitrary physical page $p$ ***securely*** across arbitrary security domains, the OS can allocate to the different domains different PPNs among the group of PPNs that merge to $p$.

**Comparison to Broader Prior Works:** While a prior work [75] also explores unconventional logic placement for security, it moves memory protection (specifically memory encryption/decryption) logic from the MC to caches to speed up decryption, whereas we do the opposite – moving cache protection near memory. While broader prior works on memory (e.g., [22, 38, 69]) and compute (e.g., RISC vs. CISC) also target reducing hardware complexity, they are outside the scope of reuse-based side channels.

This paper makes the following contributions:

- We propose a near-memory cache protection for the reuse side channel due to sharing across domains. Placing cache side channel protection near memory (i.e., in the MC) allows everything needed for global protection to come together in just one hardware component, so that ***local changes*** to the MC alone can now achieve ***global protection***.
- We propose a novel table-free merging to securely merge addresses at the MC free of any performance penalty.
- We incorporate CPM into a high-end Intel server. Evaluations show secure protection against cache reuse side channels across containers, VMs, and cgroups. CPM slightly improves performance over the mainstream practice of disabling sharing, ***while restoring memory sharing***.

## 2 Background

This section provides relevant background on memory.

### 2.1 Memory Hierarchy

Cache and DRAM are two parts of today's memory system where a shared page or block can reside. This subsection describes the relevant background on cache, memory, and memory sharing.

*2.1.1 Cache.* A cache is organized as a two-dimensional array of sets and ways, where each way is typically a 64B cacheline. For example, a 128MB cache with 16-way associativity contains 128MB/(16 * 64B) = 131072 sets. The cache set used by a memory block is determined by the block's physical address, which in turn consists of the Physical Page Number (PPN) and Page Offset. The set index is determined by several bits from both the PPN and page offset of the block (see Figure 3).

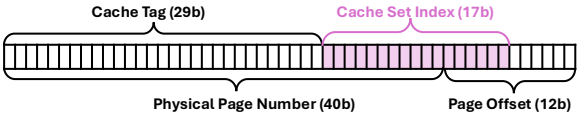



**Figure 3: Layout of the 52-bit physical address of a cacheline in a 128-MB 16-way cache, assuming physical page size is 4 KB. The cache tag (bits 23 and above) identifies which block the cacheline belongs to. Set Index (bits 6 to 22) selects which set in the cache the line maps to.**

*2.1.2 DRAM.* DRAM is organized hierarchically, with the memory controller accessing its modules via a memory channel. Each module can have one or more ranks; each rank contains multiple banks that can operate independently. Each bank contains a two-dimensional array of DRAM cells, organized into rows and columns, along with a row buffer. The address mapper in the MC translates a physical address in a memory request into a specific DRAM address, including channel, rank, bank, row, and column number (see Figure 4). The MC then encodes these numbers calculated from the physical address in the address field of DRAM commands to access

[1] Although the merging function is fixed, the page allocation can still be random as the PPNs can be randomly chosen among the fixed group of PPNs, see Section 5.6.

the memory block. When a row is accessed, its entire content is copied into the row buffer. Subsequent read or write operations are then performed from this buffer, rather than directly from the cell array, allowing fast access to data within an open row. As such, the row buffer in DRAM is effectively a cache for the most recently accessed DRAM row for each bank.

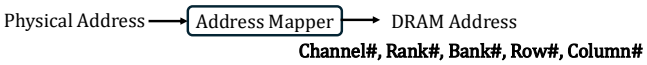



**Figure 4: DRAM address mapping. It uses a static 1:1 mapping function.**

## 2.2 Memory Sharing

The goal of memory sharing is to save memory across different programs, unlike *shared memory*, which enables communication across one program's processes and threads.

To support memory sharing, the CPU and kernel introduce the ***read/write*** status bit in PTEs. When two virtual pages are identical in value, the OS merges them by mapping both virtual pages to the same physical page and marking them as read-only in their PTEs. Afterwards, reads proceed normally; but a write to the shared page triggers a page fault during which the OS performs a copy-on-write (COW) to allocate a dedicated physical page to the written page.

There are two kinds of memory sharing.

*Allocation-time memory sharing.* This type of sharing is established at the creation time of a virtual page; the OS directly maps the newly-created virtual page to a physical page already used by another virtual page. This occurs most frequently for file-backed code/-data segments (e.g., multiple processes sharing the same loaded binary or data file), as the OS can tell the contents are identical when the accessed file names and pages are identical. The OS then protects the merged virtual pages, which can be either code or data file pages, with read-only permission.

*Memory deduplication.* This type of sharing is established after dedicated physical pages have already been allocated to the virtual pages. Examples include Linux Kernel Same-page Merging (KSM) and Windows Page Fusion. KSM is most commonly used for memory sharing across VMs; KSM running on the host OS scans for pages with identical content and merges them into a single physical page to free up redundantly allocated physical pages. Like allocation-time memory sharing, KSM protects deduplicated virtual pages with read-only permission to handle future writes via COW.

CPM can benefit both types of memory sharing.

# 3 Threat Model and Side Channels

We target a multi-tenant (e.g., cloud) scenario where victims and attackers run in different security domains (e.g., containers, VMs) while the host transparently performs allocation-time memory sharing or memory deduplication across them. The attacker mounts reuse-based side-channel attacks in software, attacking any level of the cache/memory hierarchy. We specifically focus on cross-VM memory sharing, as memory sharing within a single security domain is already considered safe. For example, cloud service providers allow memory sharing within a single VM or container [4]; similarly, VMware enables intra-VM TPS by default, while disabling cross-VM sharing by default [1].

When sharing a page across security domains, the OS sets it to **read-only** to prevent writes by one domain from corrupting another. As such, we ignore write-based software attacks.

Like all prior works on cache side channel defense (e.g., [49, 51, 60, 64, 77], while we assume attackers have full privileges in their own security domains, privilege escalation attacks that cause the host OS to malfunction (e.g., to modify access permission, to use hardware defenses incorrectly, etc) are outside of our threat model. Under escalation attacks, the attacker could directly read victim values, instead of doing so indirectly via timing side channels.

In this paper, we focus on the reuse-based side channel due to memory sharing. Different from reuse-based side channels, other side channels like conflict-based side channels and branch predictor side channels (e.g., Spectre[40]) exist regardless of memory sharing. Besides, non-intrusive and practical hardware protections already exist for them. For example, SSBD protects against the branch predictor side channel, while CAT can be used to defend against cache conflict side channel for a limited number of colocated workloads if there is no memory sharing across them (see Section 8).

## 3.1 Reuse-based Side Channels

Historically, cloud providers allow memory sharing across different VMs and containers. However, researchers later discovered that memory sharing opens up a microarchitecture side channel [19, 20, 28, 29, 40, 45, 78–80, 82]; the discovery causes cloud providers to turn off memory sharing across different VMs and containers.

Memory sharing opens up the reuse-based side channel, which exploits the reuse of shared code/data. Reuse happens when one container/VM accesses the data that another container/VM recently accessed and observes obviously shorter latency (see Figure 5). The attacker exploits the reuse side channel to detect the victim's access pattern for certain shared code/data. If the shared code/data access is indexed by a secret, the attacker can infer the victim's secret.

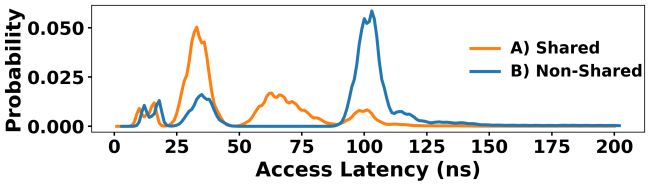



**Figure 5: Probability of different access latencies for 100 million accesses on a server with an Intel Platinum 8462Y+ CPU, same as Figure 18 except this experiment is performed for local memory instead of CXL memory. During the measurement window, two instances of the same program are accessing A) a shared file or B) different copies of the same file.**

*3.1.1* ***Reuse-based Side Channel in Cache.*** Flush+Reload [78] is the most common reuse-based cache side channel attack. In this attack, an attacker process first flushes a specific memory block (which is shared with the victim) from the CPU cache (see Figure 6). It then waits for a certain period and reloads the shared code or data to check if it has been accessed by the victim process. If the victim accesses the shared code or data, it will cache the content in the CPU cache, causing quicker access when the attacker reloads it. The victim's access pattern to shared code or data can reveal secrets (e.g., such as cryptographic keys or sensitive data) from private/nonshared pages, since the code/data in private pages can

access shared pages. For example, an attacker can infer RSA keys based on which shared addresses the victim accesses.

The victim and the attacker under Flush+Reload can run in either the same core or different cores of the same CPU since they share the same LLC, which is used by all cores.

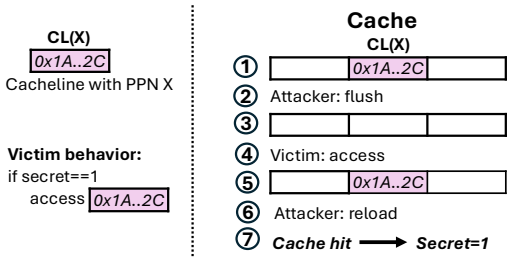



**Figure 6: An example of Flush+Reload attack. The secret of victim has one bit. The victim accesses the shared block when the one-bit secret is 1.**

*3.1.2* ***Reuse-based Side Channel in Memory.*** Memory sharing introduces a reuse-based memory side channel, where an attacker can detect a victim's memory access pattern by exploiting row buffers. Because of the caching behavior of the row buffer, similar to a CPU cache, the row cached in the row buffer can also be reused, suffering from a reuse-based side channel. Specifically, when the victim accesses a shared page, the corresponding DRAM row is activated and left open in the row buffer. Subsequent accesses by the attacker to the same shared page can result in a row buffer hit, revealing that the victim recently accessed that row [56].

## 3.2 Severity of Reuse-based Side Channels

Among various side channels, the reuse-based side channel stands out due to being alias-free, posing a particularly severe security risk. Because the side channel relies on memory shared between the attacker and victim, the attacker can directly observe whether a specific memory block was accessed by the victim just by measuring access latency, resulting in a clean high-bandwidth side channel. In comparison, conflict-based cache side channel (e.g., Prime+Probe [53]) suffers from aliasing. Specifically, conflict-based attacks exploit cache conflicts between processes, making them harder to execute successfully [81][37] [24] [35][36][45][52][59][72][74]. These attacks suffer from aliasing, revealing only whether any block in a cache set was accessed instead of whether a specific block with a specific address was accessed, resulting in a much lower side channel bandwidth, especially in noisy environments like cloud.

Because reuse-based side channel is a dangerous high bandwidth side channel [19, 20, 28, 29, 40, 45, 78–80, 82], many cloud providers disable memory sharing across VMs [7, 27, 48]. They even disable memory sharing across VMs of the same user [48] to prevent malware that successfully compromises one of the user's VMs from launching reuse-based side channel attacks against other VMs of the same user. Also, VMware disables inter-VM Transparent Page Sharing (TPS) by setting the option *Mem.ShareForceSalting* by default [16]. In comparison, cloud service providers do not take any defense measures specifically against the conflict side channel.

# 4 Related Works on Side Channel Defense

This section discusses prior hardware defenses against reuse side channels, both in caches and in memory.

**Cache Defenses:** Prior works mitigate reuse-based cache side channels by modifying the cache hierarchy [49, 51, 60, 77], which is intrusive. They modify numerous L1 caches, L2 caches, LLC slices, the coherence directories, the MMUs, as well as the OS.

One body of prior hardware defenses works by faking cache miss latencies on cross-domain reuse of shared cachelines to hide the timing differences caused by memory sharing [51][60][49][39][65]. Here, the state-of-the-art work is TimeCache [51], which fakes cache miss when a process first hits on a shared cacheline loaded into the cache by another process. TimeCache modifies all levels of caches [51] so that all caches can be protected. TimeCache modifies the OS kernel to store security bits identifying each security domain and exposes them to the caches so the protection logic can detect cross-domain cacheline reuse. Exposing per-domain security bits from kernel memory to cache requires changes to the MMU, which interfaces between the OS and the caches. TimeCache also incurs costly LLC area overhead. For example, assuming a 64-core CPU with two hardware contexts per core, TimeCache [51] can incur a 31% overhead for each 512b LLC cacheline.[2]

However, when multiple domains use the same cacheline, always securely faking no reuse under all possible scenarios is difficult. One group of scenarios missed by TimeCache are the coherence attacks (e.g., Flush+Flush [28]) which are based on remote cache reuse (e.g., when an attacker running in one core using a private cache accesses shared blocks in another core's private cache in an exclusive cache hierarchy). Faking no reuse for remote cache reuse would require changes to the coherence directory as well. Other unforeseen scenarios of cross-domain reuse may also exist.

A second body of prior works seeks to provide more comprehensive protection against cache side channels [64, 77]. They keep in the same cache multiple copies of the same shared memory block, each for a different security domain, to completely eliminate the possibility of cacheline reuse across domains. They do so by modifying the cache set indexing function to map the same shared block to different sets when the block is accessed by different security domains. Mapping numerous copies of the same shared block to different sets is more secure than mapping them to the same set, which could greatly increase set contention and worsen the conflict side channel. Since each cache level has its own set indexing function, all caches must be modified. To prevent the set-associative coherence directory from facing the same set contention problem of having many copies of the same block in the same set, the hardware set indexing function for the directory must be modified similarly. Conveying the security domain information to the caches still requires modifying the MMU and the OS.

**Memory Defenses:** Addressing the reuse side channel in memory requires accesses to a shared page behave as if the page is not shared. Specifically, MC must eliminate the timing effects caused by row buffer reuse: after one domain accesses a shared page, subsequent accesses to the same page by different domains should result

[2]Each cacheline needs a 32-bit Tc to record when it is inserted into cache and 1-bit s-bit per hardware context, so the overhead for supporting 128 hardware contexts is $(1b * 128 + 32b)/512b = 31\%$ in total.

in row buffer misses, not row buffer hits. Prior defenses [23][66][76] propose removing row buffer hits to avoid this reuse.

**Summary:** Leveraging prior defenses to comprehensively protect against reuse-based side channels across the memory hierarchy (i.e., in both cache and memory) would require modifying many places in the CPU, including the MC, LLC, L1/L2 caches, cache directory, and MMU (see Table 1). Pervasive changes across numerous places are intrusive and difficult in practice.

# 5 Main Idea

This paper explores a non-intrusive hardware protection that places all hardware necessary for achieving full-hierarchy (i.e., memory + ***all*** caches) protection against cross-domain reuse-based side channels ***together in one CPU component***. There are only two possible options to put both cache and memory protection in one place – either moving cache protection near memory or moving memory protection to cache. We take the first option as the latter is more expensive in terms of area, due to needing a copy of the hardware in each of the many private caches and LLC slices, and also in terms of performance and energy, as caches are accessed more frequently than memory.

We call our new cache protection placed remotely in the MC *Non-intrusive **C**ache Side Channel **P**rotection Near **M**emory (CPM).*

To move cache protection near memory, we observe the root cause of the cache reuse side channel is the shared physical addresses in caches. Thus, an alternative memory design that supports memory sharing without requiring shared physical addresses in caches would completely eliminate all reuse-based cache side channels and, thus, provide full-coverage guaranteed cache protection.

As such, we propose delaying the merging of unique addresses into shared addresses after requests leave the cache hierarchy (i.e., when they arrive at the MC); this differs from conventional systems where unique addresses are merged into shared addresses at the input to the cache hierarchy (i.e., by the MMU).

Specifically, CPM enhances the MC to merge unique PPNs to the same PPN. Figure 7 shows the architecture overview of CPM with an example of two security domains sharing the same page. The only hardware change is adding a PPN merging unit into the MC. When the OS allocates different PPNs ($X_1$ and $X_2$) to different security domains ($D_1$ and $D_2$), CPM merges the different PPNs to the same PPN to share memory. Because CPM enables different security domains to use different PPNs and, thus, use different cachelines while sharing memory cross the domains, CPM eliminates all cross-domain reuse-based side channels from ***every*** cache.

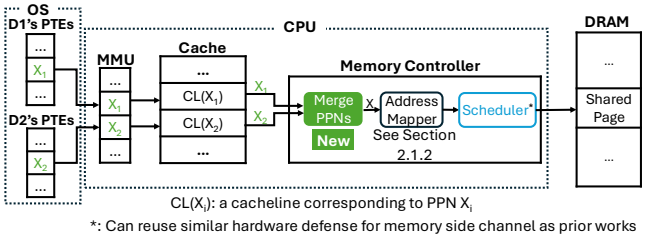



**Figure 7: CPM Architecture overview.**

## 5.1 Design Challenge and Solution

The key design challenge is that enhancing the MC to merge PPNs can introduce new address translation overheads. A practical hardware protection should incur little or no performance overhead. While executing memory instructions, existing systems merge virtual addresses to shared physical addresses by relying on the MMU to look up page tables, which record the shared PPNs. Adopting this table-based approach to perform address merging in the MC would require the MC to access a new PPN-to-PPN mapping table (see Figure 8). Address translation is already a memory performance bottleneck for real-world data center workloads, as exemplified by the extensive research on address translation [18, 47, 54, 63]. Requiring the MC to access an added table can further slow down memory accesses, as data accesses can only start after accessing the table to determine where to access. Also, accessing the table entries incurs a bandwidth overhead. Adding a translation cache to speed up the table access further incurs an SRAM area and latency overhead (i.e., adding cache hit latency per access).

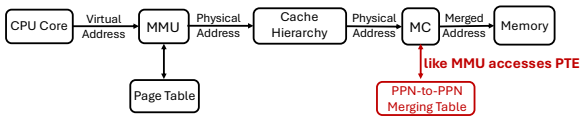



**Figure 8: Design Challenge: Reusing table-based approach for merging addresses in the MC would harm performance.**

To address the design challenge, we propose a novel table-free merging that maps each fixed group of PPNs to the same PPN (i.e., map all PPNs in Group $X$ to PPN $X$, see Figure 2). As a result, for any given input PPN, the merging unit that CPM adds to the MC can then directly calculate the output PPN, without any table access. The OS can utilize the table-free merging hardware to share memory securely across arbitrary security domains by allocating to the arbitrary security domains different PPNs from the same group.

CPM uses the following equation to define each unique PPN that merges to the same PPN X:

$$X_i = C + i + X \times S; i \in \{1, 2, \ldots, S\} \quad (1)$$

In Equation 1, $X_i$ refers to the $i^{th}$ PPN that merges to PPN X. $S$ is the group's fixed size (i.e., the total number of PPNs in the group). C is a number equal to (or greater than) the total installed memory capacity of the system in units of 4KB physical pages. Meanwhile, we also refer to the index $i$ as the group member ID.

Given a PPN $X_i$, where $X_i$ >=C, CPM can recompute the actual PPN to use for accessing memory (i.e., the post-merging PPN) as:

$$X = (X_i - C)/S \quad (2)$$

where '/' is integer division.

While Equation 1 comprises only three simple terms – $C$, $i$, $X \times S$, these three terms are carefully chosen to preserve memory space efficiency, to preserve the same cache security properties as turning off memory sharing, and to preserve correctness, respectively. Sections 5.2, 5.3, 5.4 explain the three terms, respectively; meanwhile Section 5.5 will describe choice of $C$ and $S$ values to speed up merging calculation (i.e., speed up Equation 2).

## 5.2 Preserving Space Efficiency

If not done carefully, merging multiple PPNs to the same PPN can leave some physical locations unused when merging the original PPN for accessing that memory location with another PPN, wasting memory (see Figure 9.a). To avoid this, we compose each fixed group strictly from PPNs beyond the installed memory range so that PPNs in the installed memory range always point to the same memory location as existing systems, without wasting memory (see Figure 9.b). Hence the $C$ term in Equation 1, which ensures all PPNs used for merging are always above the installed memory range.

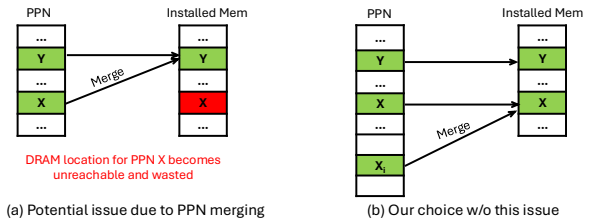



**Figure 9: Different choices of PPNs to merge; $x_i$ refers to a sharing-only PPN.**

Since we only use the PPNs beyond the installed memory range for sharing, we call them ***sharing-only PPNs***. Conversely, we call the PPNs within the installed memory range ***conventional PPNs.***

## 5.3 Preserving Other Cache Security Properties of Turning Off Memory Sharing

The construction of each group can affect cache miss rate and, thus, security. Consider a naive construction where a sharing-only PPN $X_i$ is simply PPN X plus putting i in some unused most significant PPN bits; constructing a group of sharing-only PPNs this way means the PPNs in the group will differ only in their Cache Tag bits instead of the Cache Set Index bits (see Figure 3). This in turn causes all cached copies of the same memory block in a shared physical pages to map to the same set. In comparison, in mainstream deployment, which turns off memory sharing for security, different security domains are allocated unrelated PPNs that map to unrelated cache sets. Mapping all cached copies of the same shared memory block into the same set would alter and could weaken cache security properties relative to mapping them to unrelated sets.

Consider, for example, some shared code vulnerable to the conflict side channel; when knowing other cached copies of the same shared code used by victims will also use the same cache set as the attacker's cached copy of the code, an attacker can focus all time and effort on surgically creating and monitoring a conflict set involving the attacker's copy of the code. Compared to creating and monitoring many cache sets, focusing on one or a few cache sets can greatly amplify conflict side channel bandwidth; this would be unfortunate because cloud currently "handles" the conflict side channel by banking on its bandwidth being too low to be of concern under the noisy cloud execution environment.

The small $+i$ term in our Equation 1 ensures that the same memory block across the different sharing-only PPNs in a group will always use different sets, just like when turning off memory sharing. Because $i$ ranges over all integers in $[0, S)$ and each value of $i$ is for a unique sharing-only PPN within the same group, the PPNs in a group necessarily differ in their lowest $\log_2 S$ bits (see Figure 10). Since the lower bits determine the cache set index (Figure 3), cachelines with different sharing-only PPNs naturally map to different sets ***without requiring any changes to the cache logic***.

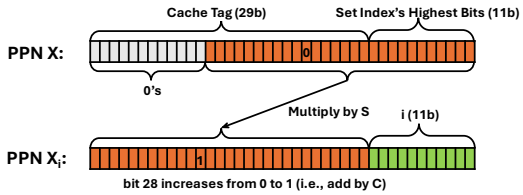



**Figure 10: An example of calculating a 40-bit sharing-only PPN for a system with 1TB memory. Bits are ordered from most significant (left) to least significant (right). Here $S = 2^{11}$, $C = 2^{28}$ and $i$ has 11 bits.**

To ensure different security domains use *unrelated* (not just *different*) cache sets when sharing a physical page under CPM, the OS may randomly choose unused sharing-only PPNs from a group X to allocate when sharing physical page X across domains.

## 5.4 Preserving Correctness

Different groups of sharing-only PPNs, used for different conventional PPNs, should be strictly disjoint. Otherwise, incorrect sharing could occur, potentially leading to silent data corruption.

The $X \times S$ in Equation 1 is to ensure no overlap between groups.

A related question is: What happens when all (i.e., the $S$) sharing-only PPNs in a group X run out? Correctness will not be affected. This simply means CPM can no longer securely share the same PPN X with yet another additional security domain. However, the content of PPN X can be copied to a new physical page Y (e.g., by the OS); another $S$ new domains can then securely share PPN Y.

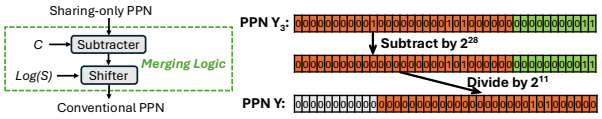
Running out of sharing-only PPNs in a group is rare because $S$ can be quite large. To compute the group size $S$, we note that the biggest sharing-only PPN (i.e., $C + C \times S - 1$) should be less than $T$ (the total possible number of PPNs in current systems). In other words, we require $S < T/C$. For example, assuming a 4KB page size and the 4PB physical address space being used today (see Section 5.5 in Intel Architectures Software Developer's Manual [34]), $T = 2^{52}/4\,\text{K} = 2^{40}$. For a 1 TB memory system, $C = 1TB/4\,\text{KB} = 2^{28}$; as such, $S < T/C = 2^{40}/2^{28} = 2^{12}$. In other words, the total number of sharing-only PPNs in a group is at most $2^{12} - 1$; so many security domains having the same page content would be extremely rare.

## 5.5 Circuit for Table-free Merging

CPM performs address merging for the PPN received in a memory request if the PPN is a sharing-only PPN (i.e., the PPN is $>= C$).

CPM uses Equation 2 to compute the merged PPN to use to access DRAM. Choosing good values of $S$ and $C$ in Equation 1 (and thus in Equation 2) can simplify and speed up the computation of the merged address. The hardware overhead of the integer division in Equation 2 can be reduced by choosing a $S$ value that is the nearest power-of-2 smaller than the maximum-allowable $S$ value; this can reduce the division to a right shift. For example, for a 1TB system, $S$ can be chosen to be $2^{11}$, which is the biggest power of 2 smaller than $2^{12}$. Conversely, choosing a $C$ value that is the nearest power-of-2 greater than the total number of installed physical pages in the system can both shorten carry propagation and simplify hardware.

The hardware circuit for table-free merging contains one subtractor and shifter (see Figure 11a). Figure 11b shows an example of the PPN merging that reverses the calculation in Figure 10.




(a) Merging logic. (b) An example of merging PPN.

**Figure 11: A) PPN merging circuit and B) an example of the merging calculation. The example assumes the same system as Figure 10.**

## 5.6 Security Analysis

**No Cacheline is Reused Across Domains:** When turning off sharing, the OS allocates different PPNs to different security domains to prevent cacheline reuse across domains. CPM preserves this property: OS assigns different security domains distinct PPNs, preventing cacheline reuse. Below, we prove that Equation 1 ensures sharing-only PPNs assigned to different live domains are never equal regardless of whether they belong to the same group.

Suppose security domain A has a sharing-only PPN $X_i$. If security domain B has a sharing-only PPN $X_j$ in the same group:

$$X_i - X_j = (C + X \times S + i) - (C + X \times S + j) = i - j.$$

Because $i \neq j$, $X_i \neq X_j$.

Now, suppose B has a sharing-only PPN $Y_j$ in another group ($i$ could be either equal or not equal to $j$):

$$X_i - Y_j = (C + X \times S + i) - (C + Y \times S + j) = (X - Y) \times S + (i - j).$$

Because $X \neq Y$, we know $|X - Y| \geq 1$ and thus $|(X - Y) \times S| \geq S$. Also, because $0 \leq i, j < S$, $|i - j| < S$. As such, $X_i - Y_j = (X - Y) \times S + (i - j) \neq 0$; therefore, $X_i \neq Y_j$.

**Cache Conflicts Same As Turning off Sharing:** When turning off sharing, a virtual page can use any physical page. However, for a virtual page in domain A to securely share a physical page P currently used by another domain B requires the virtual page in A to only use one of the sharing-only PPNs in group P. This reduction in choice of PPNs can seem to worsen cache conflict rates and, thus, worsen the conflict side channel.

This is not the case, however, because the possible cache sets a virtual page could use – depending on which PPN is allocated to the page – is limited. As long as the limit is smaller or equal to group size $S$, there is no reduction in freedom of page coloring.

In current caches, the 64 blocks in a 4KB page can span 64 sets. We refer to 64 sets as a set collection. Each cache has a limited number of sets and, thus, a limited number of possible set collections for a 4KB virtual page. For example, a 16-way 64MB LLC has $64MB/(16 \times 64B)/64 = 1024$ set collections. As such, a virtual page using only one of the 1024 sharing-only PPNs in a group can still end up using any of the 1024 possible set collections, just like when turning off sharing where a virtual page can use any free PPN in the system.

**Other Side Channels Related to Memory Sharing:** Memory deduplication (as opposed to allocation-time memory sharing) also introduces a COW side channel; however, the COW side channel can be efficiently addressed by software-only defenses [43] and, thus, need not be re-discussed in our paper.

## 5.7 Software Complexity Analysis

The OS changes to use CPM for memory sharing across security domains is small – the change is to always allocate sharing-only PPNs when sharing a physical page with another domain, instead of directly allocating the shared physical pages' original conventional PPNs. When sharing, the OS can allocate any free sharing-only PPN in a group to any security domain; there need not be any affinity for a specific domain (e.g., domain 1) to use a specific sharing-only PPN (e.g., always use the first sharing-only PPN per group).

Note prior reuse side channel defense (see Section 4) also need some OS changes. In general, disabling cacheline reuse selectively only for cross-domain reuse (as opposed to all reuse) requires the OS to pass some domain identifying information to hardware, making some software changes unavoidable.

Comparatively speaking, the software overheads to use CPM are no more than ScatterCache [77] and Mirage [64]. These prior works install a Security-domain ID (SDID) in every page table entry, effectively tapping into every page allocation and deallocation, whereas the OS changes for CPM only taps into allocation and deallocation *due to sharing and unsharing*.

CPM offers one software advantage over the prior works, however. Under CPM, the OS works strictly with existing OS-MMU interfaces; allocating sharing-only PPNs is no different from allocating conventional PPNs. However, conveying the SDIDs in prior works to hardware requires the OS to work with some customized/extended OS-MMU interface because today's CPUs only support hardware security domains for the purpose of memory encryption [10, 12, 30, 50]. Under many (e.g., ARM, IBM Power, AMD SEV) ISAs, the current encryption security domains are hardcoded to VMs at the level of the virtualization hardware; as such, their current hardware cannot differentiate containers and host-level processes, unless some hardware changes are made. Furthermore, per-VM memory encryption individualized to each VM would also preclude memory sharing across VMs; as such, the memory encryption domains under the various ISAs cannot generally be repurposed to manage security domains for reuse side channel protection without making further changes to the OS-MMU interface.

## 5.8 Generality

**Multiple MCs:** Modern CPUs may interleave the memory blocks of a physical page across MCs. For example, the CPU may route the even blocks within a page to MC0 and route the odd blocks within a page to MC1. Because CPM preserves the page offset for every block, CPM is compatible with the fine-grained inter-channel page interleaving. Each MC merges the sharing-only PPNs only for the memory requests it receives (e.g., MC0 merges the PPN $X_i$ to $X$ for requests to even-indexed blocks with PPN $X_i$ while MC1 merges the PPN $X_i$ to $X$ for requests to odd-indexed blocks with PPN $X_i$.) In short, Equations 1 and 2 naturally support multiple MCs.

**Multiple Sockets:** Today, some cloud servers contain up to two sockets/CPUs with a shared global physical memory address space; this global range is partitioned into two continuous address ranges, one for each socket. CPM can work as follows. A) Divide the total possible physical address space evenly into two ranges – $[0, T/2)$ and $[T/2, T)$ – for socket 1 and socket 2, respectively. B) For socket 1, we can extend Equation 1 to define the sharing-only PPNs to

ensure they are within $[0, T/2)$:

$$X_i = (C/2) + X \times S + i. \quad (3)$$

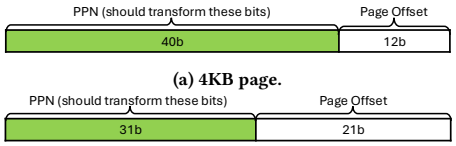
C) For socket 2, we can extend Equation 1 to define the sharing-only PPNs to ensure they are all within [T/2, T):

$$X_i = (T/2) + (C/2) + (X - T/2) \times S + i. \quad (4)$$

The group size $S$ in Equations 3 and 4 remains unchanged compared to a single-socket system with the same amount of installed memory capacity (i.e., $S < (T/2)/(C/2)$, which is the same as $S < T/C$).

**Huge Pages:** In this paper, we focus on 4KB pages because current systems typically only support sharing at the 4KB granularity instead of sharing 2MB huge pages. However, CPM can be extended to huge pages with minimal changes.

When merging PPNs for huge pages, the MC should merge only the most significant bits of the requested address, since huge-page PPNs consist of fewer bits than those of 4KB pages (see Figure 12). As such, the MC must first determine the page size of each memory request; one way to do this is to use different address ranges for 4KB and huge pages, so that the page size of a request can be conveyed simply through the request's address range.




**(a) 4KB page.**

**(b) 2MB page.**

**Figure 12: The physical address bits to transform for 4KB and 2MB pages.**

We extend Equation 1 as:

$$X_i = EC + X \times S + i. \quad (5)$$

where $EC = C$ for 4KB pages, but $EC = (C + C \times S)/(2MB/4KB)$ for huge pages. For example, for a server with 1TB memory, we let $S = 2^{10}$. When the physical page is a 4KB page, $EC = C = 2^{28}$; when the page is a 2MB page, $EC = (C+C\times S)/(2MB/4KB) = 1025\times 2^{19}$. In this way, the sharing-only addresses of 4KB pages locate at $[1T, 1025T)$ while sharing-only addresses of huge pages always locate at $[EC \times 2M, EC \times 2M + EC \times 2M \times 2^{10}) = [1025T, 2049T)$. Correspondingly, we extend the Equation 2 for calculating conventional PPN as:

$$ConventionalPPN = (SharingOnlyPPN - EC)/S. \quad (6)$$

**Applicability to Conflict Side Channel Protection:** Like many prior cache defenses [49, 51, 60, 77], our paper focuses exclusively on the cache reuse side channel. However, CPM may also be used to help reduce conflict side channel bandwidth.

Conflict side channel defenses generally work by randomly changing which sets are used by the same virtual pages. The OS can do this by randomly migrating virtual pages to new physical pages [15]; however, OS-enacted page migration is costly, resulting in infrequent migration and causing some conflict side channel to remain. Prior hardware defenses that also protect against conflict side channels (e.g., ScatterCache [77] and Mirage [64]) work by randomly migrating data in hardware, which is cheaper and, thus, can be much more frequent, lowering the conflict side channel bandwidth. Similarly, CPM can also enable cheaper and, thus, more frequent migration of virtual pages to different sets by allowing the OS to randomly remap a virtual page to another PPN in the same group *without paying the cost of physically copying its data to another physical page*. While CPM does not support subpage-level set migration/randomization like ScatterCache and Mirage, their individualized per-domain set composition also means each row in an SRAM bank often does not contain all ways of a customized set; as such, each lookup to an N-way cache requires accessing ~N SRAM banks, presenting a different tradeoff in cost vs. security.

We will leave a detailed exploration of generalizing CPM to protect against the cache conflict side channel for future work.

## 6 Methodology

**Figure 13: Real-system prototype of CPM.**

We prototype CPM in a high-end server with a 32-core Intel EMR Gold 6530 CPU (see Figure 13). The server includes an FPGA board that implements a Compute Express Link (CXL) [2] memory controller. Since the integrated memory controllers (IMCs) for accessing local memory are unmodifiable, we implement CPM within the FPGA-based CXL memory controller (see Figure 14). To clarify, CPM is orthogonal to CXL memory, and applicable to all kinds of memory; we use the FPGA-based CXL memory strictly to prototype and evaluate CPM. As such, our evaluation just accepts CXL memory's higher latency over host memory [46] without any CXL-related optimizations like migrating hot pages to local memory and cold pages to CXL memory; when running an application, we use numactl to bind the entire application memory to CXL.

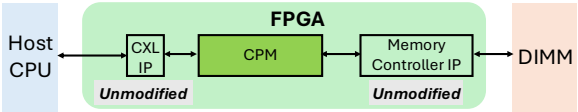



**Figure 14: Overview of the real-system implementation of CPM.**

To support the sharing-only PPNs, which exceed the range of installed memory, we configure the CXL memory controller to expose 512 GB to the OS, even though the FPGA board has only 16GB of DRAM installed onboard. To ensure the OS only uses the upper 496 GB (addresses 16-512 GB) of the exposed CXL physical memory range for sharing-only PPNs, we use the Linux memmap GRUB boot option [57] to reserve the range for use when allocating a shared physical page across different security domains. Each security domain in our prototype refers to a Linux *Cgroup*. A Cgroup allows the host OS to manage hardware resources such as CPU time, system memory, and network bandwidth for each job, be it a container [61], VM [62], microVM [8], or host process.

### 6.1 Software Evaluated

We run workload co-location experiments using Docker containers, Linux VMs, and host processes in separate Cgroups. The applications include OpenSSL [5], Large Language Models [73] (deployed locally, using models Llama3.2 [71], Gemma3 [70], and

Qwen2.5 [13]), DNN-based image classification [41], TPC-H queries [58] to SQLite database [26], SPEC CPU 2017 [17] and PARSEC [14]. For LLM, we send prompts that span lifestyle, finance, geography, and coding topics. For TPC-H, we run four workloads separately, with each workload executing five or six different queries, covering the diversity of query patterns.

Linux handles allocation-time sharing, which is commonly applied across host processes and containers, in mm/memory.c. Linux handles memory deduplication, which is commonly applied across VMs, in mm/ksm.c. As such, to evaluate sharing across these three types of workloads, we modify mm/memory.c and mm/ksm.c to record sharing-only PPNs into PTEs when sharing across Cgroups and also to track usage of sharing-only PPNs when these modules unshare them (see Figure 15).

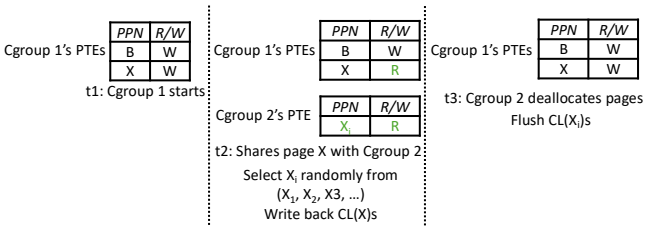



**Figure 15: Sharing-only PPN allocation and deallocation examples. Here, $X$ is a conventional PPN, while $X_i$ is a sharing-only PPN that merges to $X$. The CLWB and CLFLUSH will be explained in Section 7.2.**

Together with the lines of code added for data structure modifications, header files, makefiles etc, ~900 new lines of code are added. We create a kernel patch and release it in our artifact website [6], which also includes the System Verilog for CPM. For the ***same breadth*** of software evaluations, the needed software changes are the same as or fewer than those of prior works (see Section 5.7).

## 6.2 Overheads

**Hardware Logic Overhead:** The hardware circuit for CPM requires only a few bitwise operations (see Figure 11a). ASIC synthesis using Synopsys [68] and the 7nm tech node reports 0.2 ns latency and 0.0000667 $mm^2$ area. On FPGA synthesis, CPM fits within the existing timing slack of the default 300 MHz clock rate without needing a new pipeline stage.

**Memory Overhead:** While CPM incurs no memory overhead in hardware (i.e., the MC need not store any metadata in memory), using CPM requires the OS to pay a small memory overhead that grows O(n) with the total number of sharing-only PPNs in-use. In existing systems, the OS needs some metadata (a.k.a. a page struct) for every PPN to manage the PPN (e.g., to track whether it is in use and, if so, by what). Similarly, when using CPM, the OS tracks which Cgroup is using each sharing-only PPN; the OS ***dynamically*** allocates a 4B metadata entry when allocating a new sharing-only PPN to track its usage (see Figure 16). Even when billions of sharing-only PPNs are in use, this memory overhead is reasonable due to still allowing a net memory savings of approximately $4096 - 8 = 4088$ bytes per additional shared page over disabling sharing for security. Because the metadata is dynamically allocated whenever allocating sharing-only PPNs across more Cgroups, the total system-level memory overhead reduces down to nearly nothing (i.e., only 0.2%) when no pages are shared.

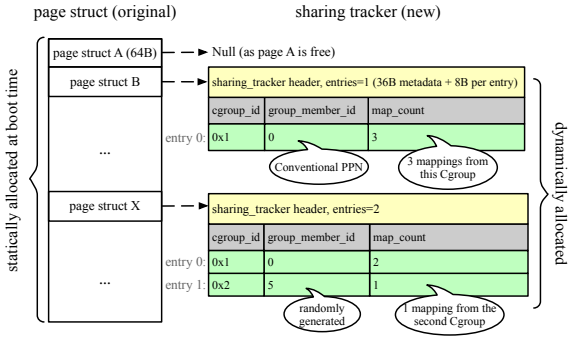



**Figure 16: A snapshot for Figure 15 t2 of the management data structure for sharing-only PPNs. For each struct page used by Linux to manage a physical page, we dynamically allocate a sharing tracker if the page is in use. Each sharing tracker starts with an 8B metadata entry. When sharing the physical page with more Cgroups, more entries are created to track the sharing-only PPNs allocated to the newer Cgroups. The group_member_id field in an entry tracks which sharing-only PPN in the group is allocated to the Cgroup recorded in the entry. The map_count field records how many virtual pages in the Cgroup are using the sharing-only PPN; the metadata entry is freed if this reduces to 0 after a page deallocation, thus, freeing the sharing-only PPN.**

# 7 Evaluation

We use our server in Figure 13 to perform the following evaluations.

## 7.1 Security Evaluation

**Protection Against Flush+Reload:** We use a publicly-available Flush+Reload code [3]. The victim and attacker mmap the same file to an array. The array has 1024 128B elements. The victim uses a secret value as the array index to repeatedly access one of the 1024 elements; w.l.o.g., we use *512* as the secret in the victim. The attacker code flushes its array and then measures the latency to re-access each array element; the code reports the array index of the fastest element as the victim's secret. The attacker repeats this over the entire array 100 times to perform 100 attack trials.

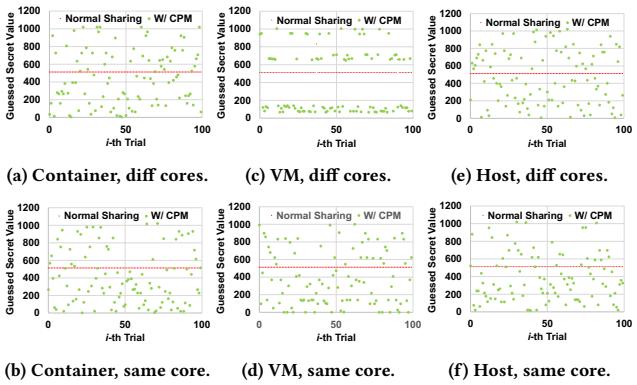
First, we run the attacker and victim in containers and pin them to different physical cores. Initially, the attack consistently reports the correct secret of *512* (see Figure 17a). But when we repeat the attack after turning on CPM on the server, the attacker consistently reports incorrect values, showing CPM securely protects against the cache reuse side channel.

Second, to verify whether CPM can provide full protection across the cache hierarchy (i.e., including L1 cache), we repeat the experiments by pinning attacker and victim to the same physical (but different logical) cores. Again, the attack consistently reports the correct secret ***except*** when we turn on CPM (see Figure 17b).

Next, we repeat the above different-core and same-core attack experiments by running the attacker and victim on different VMs. Even though two programs technically mmap different files within the different VMs, because the file contents are the same, Linux KSM deduplicates their memory pages and maps them to the same physical pages. Figures 17d and 17c show the same behavior. The attack reliably succeeds ***except*** when we turn on CPM.

Lastly, we repeat the same experiments by running attacker and victim directly on the host in separate Cgroups. Figures 17f and 17e show the same results.

**Timing Distribution:** In current systems, the timing distribution of a program's memory accesses can vary depending on

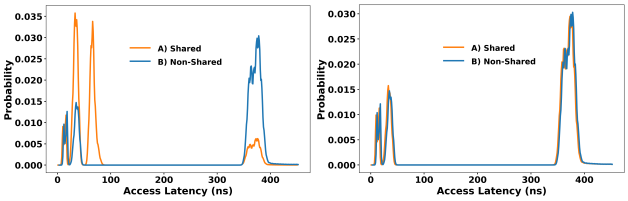



(a) Container, diff cores. (c) VM, diff cores. (e) Host, diff cores.

(b) Container, same core. (d) VM, same core. (f) Host, same core.

**Figure 17: Defense experiments against Flush+Reload attack.**

whether another program is accessing the same pages (due to memory sharing) or only accessing its own unique physical pages. For example, we run two instances of the same random-array read program in two containers. The program populates the array also by mmaping a file. In experiment A), the two container instances mmap the same file and, thus, will share memory. In experiment B), the two container instances mmap two different files and, thus, will not share memory. The LLC is big enough to cache one file, but not both files. Figure 18a shows the timing distribution across 100 million accesses; the two experiments show obviously different timing distributions. This difference in timing distribution theoretically poses a timing side channel.




(a) Without CPM. (b) With CPM.

**Figure 18: Memory access timing distribution.**

To evaluate the impact of CPM on timing distribution, we repeat experiment A) after turning on CPM. Figure 18b shows that the timing distributions for with and without sharing become indistinguishable. We also conduct a Kolmogorov-Smirnov test on 100,000 access events to compare the timing distributions of accesses with and without sharing under CPM. The resulting high p-value of 0.83 indicates no statistically significant difference between the two distributions, closing the reuse-based timing side channel.

## 7.2 Performance Evaluation

For performance evaluation, we first compare against mainstream data center deployment, which turns off memory sharing since memory sharing in conventional systems is insecure. We run one workload per container. For sensitivity analysis, we run two to four containers. With memory sharing turned off, the limited 16GB CXL cannot fit more than four containers for several of the evaluated workloads. We measure the average execution time of each workload (e.g., the four workloads for the 4-container experiments) when comparing performance. Additionally, we repeat every experiment three times to compute the average execution time.

Figure 19 shows performance when sharing via CPM normalized to turning off sharing. Compared to turning off sharing, CPM improves performance by 2.2% on average across all the comparisons. Turning off memory sharing reduces performance due to requiring the OS to load same values into duplicate memory locations.

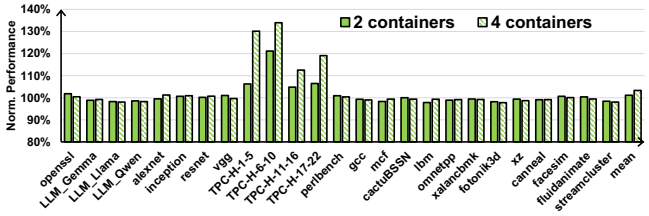



**Figure 19: Performance of CPM normalized to turning off memory sharing.**

For a few benchmarks, CPM is < 2% slower. CPM can sometimes slow down performance due to software overhead added for page allocation and deallocation to use and manage sharing-only PPNs. The biggest software overhead is to ensure the multiple cached copies of the same physical pages due to the use of sharing-only PPNs have the same values since hardware cache coherence is not enforced across the multiple cached copies of the same memory block. In our prototype, the kernel ensures the multiple cached copies have the same value by A) using the CLWB instruction to writeback a page's value to DRAM when the physical page transitions from non-shared to shared and B) using the CLFLUSH instruction to flush all blocks from a sharing-only PPN when freeing it. These actions are illustrated in Figure 15. These overheads are not needed when allocating unique pages.

Our evaluation shows that CPM increases the allocation time of shared pages by 0.3 µs per page; this is small when amortized across program lifetime. To evaluate the worst-case overheads due to the added page allocation and deallocation overheads, we run a microbenchmark that does nothing but continuously maps and unmaps every shared page of a 5GB shared file. The file is shared by four instances of the same microbenchmark. The average microbenchmark performance is 25% lower under sharing with CPM compared to normal sharing. However, the performance degradation reduces down to only 10% when increasing the number of microbenchmark instances to 8. This is because the overhead of CLWB, which is needed only when transitioning from non-shared to shared, is better amortized when sharing across 8 instances than 4 instances.

Note that software flushing overheads are also required by prior works that keep multiple cached copies of the same shared physical address; for example, ScatterCache [77] also mentions flushing overheads when allocating pages.

Compared to normal memory sharing, which is insecure and turned off by data centers, CPM can incur higher cache miss rates due to only sharing in memory, but not in cache (note, however, that prior hardware defenses like ScatterCache also only share in memory, but not in cache). We measure and compare the LLC MPKI under normal sharing and under CPM. Figure 20 shows most benchmarks' LLC MPKI increases by < 0.2 compared to turning on

memory sharing. While the LLMs suffer the worst increase in LLC MPKI, they only suffer a 10% performance degradation due to the highly parallel nature of the memory accesses.

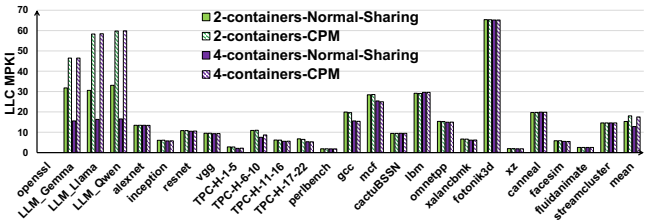



**Figure 20: LLC MPKI of CPM compared to normal sharing.**

## 7.3 Sensitivity Analysis

As a sensitivity analysis, we evaluate one way to utilize the memory saved from sharing. The saved memory can be used to cache more file pages to speed up the file system, can be used to keep more waiting/idling serviceless services 'warm' in memory, or can be used to actively run more workloads under the same physical memory capacity limit. We evaluate the latter in this section.

To broaden the spectrum of evaluations for the purpose of sensitivity analysis, we also evaluate VMs instead of containers. Each VM runs one instance of a workload.

Figure 21 compares how many workloads can run concurrently using the 16GB CXL memory when turning off sharing and when incorporating CPM into KSM to share memory across the VMs. The former only allows five VMs on average, while the latter allows 13.

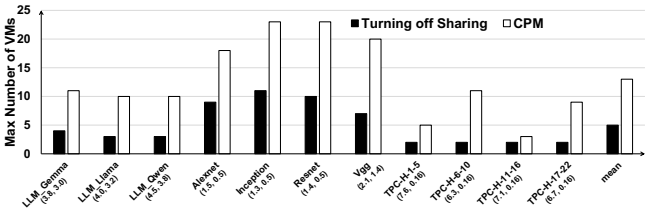



**Figure 21: Comparison on the maximum number of VMs co-located on the 16GB CXL memory. The values in '()' are the Resident Set Size (RSS) in GB of the VM and the RSS of the benchmark in the VM, respectively.**

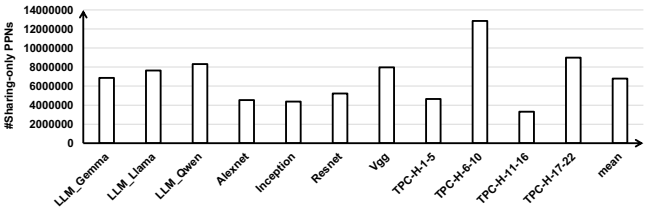



**Figure 22: The maximum number of sharing-only PPNs allocated for LLM, DNN, and SQLite when running the maximum number of VMs under CPM.**

Figure 22 shows the total number of sharing-only PPNs allocated during the CPM experiments in Figure 21. Each sharing-only PPN allocated saves 4KB of memory. The many sharing-only PPNs explain why more VMs can be co-located.

# 8 Other Related Works

Some prior works [44][25] use Intel's cache allocation technology (CAT) [33] to address side channels in the LLC. By inserting cachelines of different processes into independent cache ways, CAT prevents cache eviction between different users. But if a memory block is shared, CAT inserts only one copy of the block in the LLC in the first-touched cache partition. Enhancing CAT to insert duplicate copies of the same physical memory block would require complex changes to the cache coherence protocol. Even after the complex change, the maximum number of ***non-overlapping*** cache partitions is limited by the LLC associativity. For example, an 11-way LLC can support at most 11 non-overlapping partitions.

Prior works [21, 55, 67] have proposed hardware deduplication to enhance the MC to transparently deduplicate memory in hardware; the goal is to deduplicate more than the OS, unlike our goal of designing a ***practical*** hardware protection.

The difference in goals leads to vastly different designs. To achieve the minimum hardware modification needed to protect against the reuse side channel, CPM's ***entire*** hardware modification is adding a hardware-friendly stateless merging logic to the MC, retaining the role of identifying and deciding what to share ***with the OS***. To deduplicate more values, hardware memory deduplication identifies and decides ***in hardware*** which blocks, instead of pages, to deduplicate. This requires adding complex hardware logic to manage large data structures that consume a large (e.g., 8% [21] to 12.5% [55]) fraction of DRAM, significantly impacting performance (e.g., needing fine-grained block-level translation per access). Furthermore, the hardware-managed data structures are statically-sized (including the various exception regions [21, 55]) and are, thus, vulnerable to overflow and require complex, undescribed OS support for correctness. The static structures also cap the deduplication ratio at 16X [67], unlike CPM (i.e., can scale to $> 1000X$). Lastly, hardware deduplication also only specifically targets deduplication, unlike CPM, which is applicable to both allocation-time sharing and deduplication.

# 9 Conclusion

Prior works on hardware defenses against reuse-based side channels require modifying multiple components throughout the CPU. To design a practical protection that modifies a minimum number of components, we observe that only hardware using shared addresses needs protection; thus, reducing the number of components that use shared addresses can minimize the changes required.

As such, we propose Non-intrusive Cache Side Channel Protection Near Memory (CPM) to delay merging into shared addresses until the very end – when the request reaches memory; CPM reduces the total number of hardware components to modify for protecting against both cache and memory reuse-based side channels down to just one – the memory controller.

We implement a real-system prototype of CPM to verify whether it is indeed practical. The prototype is the *very first* real-system prototype of hardware protection against the reuse side channel. Our real-system evaluations show it securely protects against reuse attacks while sharing memory and matching the performance of disabling memory sharing across jobs (i.e., the current practice in data centers out of security concerns).

# References


[1] [n. d.]. Additional Transparent Page Sharing management capabilities and new default settings. https://knowledge.broadcom.com/external/article?legacyId=2097593 [Online; accessed Sep. 2025].

[2] [n. d.]. Compute Express Link. https://www.computeexpresslink.org

[3] [n. d.]. Hands-On Hardware Side Channels Lab. https://github.com/MATCHA-MIT/SHD-CacheAttackLab.git [Online; accessed Aug. 2025].

[4] [n. d.]. How Amazon ECS manages CPU and memory resources. https://aws.amazon.com/blogs/containers/how-amazon-ecs-manages-cpu-and-memory-resources [Online; accessed Sep. 2025].

[5] 2025. OpenSSL Library. https://www.openssl.org/. Accessed: 2025-Nov.

[6] 2026. CPM Project Website. https://698cc6af763d6d057290a1df--seam-proj.netlify.app/. Accessed: 2026-02-25.

[7] Amazon. [n. d.]. The EC2 approach to preventing side-channels. https://docs.aws.amazon.com/whitepapers/latest/security-design-of-aws-nitro-system/the-ec2-approach-to-preventing-side-channels.html?

[8] Amazon. 2020. Logical Separation on AWS: AWS Whitepaper. https://docs.aws.amazon.com/pdfs/whitepapers/latest/logical-separation/logical-separation.pdf#serverless-and-containers

[9] Amazon. 2022. Security Overview of AWS Lambda. https://docs.aws.amazon.com/pdfs/whitepapers/latest/security-overview-aws-lambda/security-overview-aws-lambda.pdf

[10] AMD. 2020. Strengthening VM isolation with integrity protection and more. (2020). https://www.amd.com/content/dam/amd/en/documents/epyc-business-docs/white-papers/SEV-SNP-strengthening-vm-isolation-with-integrity-protection-and-more.pdf

[11] AMD. 2026. AMD CPU Timing and Power Side Channel Guidance. https://docs.amd.com/api/khub/documents/tWeDJc6ue~luJn6X3fJktA/content?. Accessed: 2026-04-08.

[12] Arm Limited. 2021. *Learn the Architecture: TrustZone for Armv8-A*. Version 1.0, Issue 02 102418. Arm Limited. https://developer.arm.com/-/media/Arm%20Developer%20Community/PDF/Learn%20the%20Architecture/TrustZone%20for%20Armv8-A.pdf Accessed: 2026-04-07.

[13] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. 2023. Qwen technical report. *arXiv preprint arXiv:2309.16609* (2023).

[14] Christian Bienia, Sanjeev Kumar, Jaswinder Pal Singh, and Kai Li. 2008. The PARSEC benchmark suite: Characterization and architectural implications. In *Proceedings of the 17th international conference on Parallel architectures and compilation techniques*. 72–81.

[15] Ferdinand Brasser, Srdjan Capkun, Alexandra Dmitrienko, Tommaso Frassetto, Kari Kostiainen, and Ahmad-Reza Sadeghi. 2017. DR. SGX: hardening SGX enclaves against cache attacks with data location randomization. *arXiv preprint arXiv:1709.09917* (2017).

[16] BROADCOM. [n. d.]. Security considerations and disallowing inter-Virtual Machine Transparent Page Sharing. https://knowledge.broadcom.com/external/article/318959/security-considerations-and-disallowing.html [Online; accessed Sep. 2025].

[17] James Bucek, Klaus-Dieter Lange, and Jóakim v. Kistowski. 2018. SPEC CPU2017: Next-generation compute benchmark. In *Companion of the 2018 ACM/SPEC International Conference on Performance Engineering*. 41–42.

[18] Siyuan Chai, Jiyuan Zhang, Jongyul Kim, Alan Wang, Fan Chung, Jovan Stojkovic, Weiwei Jia, Dimitrios Skarlatos, Josep Torrellas, and Tianyin Xu. 2025. {EMT}: An {OS} Framework for New Memory Translation Architectures. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*. 711–729.

[19] Guoxing Chen, Sanchuan Chen, Yuan Xiao, Yinqian Zhang, Zhiqiang Lin, and Ten H Lai. 2019. Sgxpectre: Stealing intel secrets from sgx enclaves via speculative execution. In *2019 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 142–157.

[20] Yun Chen, Lingfeng Pei, and Trevor E Carlson. 2023. AfterImage: Leaking control flow data and tracking load operations via the hardware prefetcher. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 16–32.

[21] David Cheriton, Amin Firoozshahian, Alex Solomatnikov, John P Stevenson, and Omid Azizi. 2012. HICAMP: architectural support for efficient concurrency-safe shared structured data access. *ACM SIGPLAN Notices* 47, 4 (2012), 287–300.

[22] Victor Costan, Ilia Lebedev, and Srinivas Devadas. 2016. Sanctum: Minimal hardware extensions for strong software isolation. In *25th USENIX Security Symposium (USENIX Security 16)*. 857–874.

[23] Peter W Deutsch, Yuheng Yang, Thomas Bourgeat, Jules Drean, Joel S Emer, and Mengjia Yan. 2022. DAGguise: mitigating memory timing side channels. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 329–343.

[24] Craig Disselkoen, David Kohlbrenner, Leo Porter, and Dean Tullsen. 2017. {Prime+ Abort}: A {Timer-Free}{High-Precision} L3 Cache Attack using Intel {TSX}. In *26th USENIX Security Symposium (USENIX Security 17)*. 51–67.

[25] Xiaowan Dong, Zhuojia Shen, John Criswell, Alan L Cox, and Sandhya Dwarkadas. 2018. Shielding Software From Privileged {Side-Channel} Attacks. In *27th USENIX Security Symposium (USENIX Security 18)*. 1441–1458.

[26] Kevin P Gaffney, Martin Prammer, Larry Brasfield, D Richard Hipp, Dan Kennedy, and Jignesh M Patel. 2022. SQLite: past, present, and future. *Proceedings of the VLDB Endowment* 15, 12 (2022).

[27] Google. [n. d.]. 7 ways we harden our KVM hypervisor at Google Cloud: security in plaintext. https://cloud.google.com/blog/products/gcp/7-ways-we-harden-our-kvm-hypervisor-at-google-cloud-security-in-plaintext?m=1

[28] Daniel Gruss, Clémentine Maurice, Klaus Wagner, and Stefan Mangard. 2016. Flush+ flush: A fast and stealthy cache attack. In *International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*. Springer, 279–299.

[29] David Gullasch, Endre Bangerter, and Stephan Krenn. 2011. Cache games–bringing access-based cache attacks on AES to practice. In *2011 IEEE Symposium on Security and Privacy*. IEEE, 490–505.

[30] Intel. [n. d.]. Intel® Architecture Memory Encryption Technologies. https://cdrdv2-public.intel.com/679154/multi-key-total-memory-encryption-spec-1.4.pdf

[31] Intel. 2023. Configuring Workloads for Microarchitectural and Side Channel Security. https://www.intel.com/content/www/us/en/developer/articles/technical/software-security-guidance/best-practices/securing-workloads-against-side-channel-methods.html. Accessed: 2026-04-08.

[32] intel. 2025. GitHub - intel/intel-cmt-cat: User space software for Intel(R) Resource Director Technology. https://github.com/intel/intel-cmt-cat/tree/master

[33] C Intel. 2015. Improving real-time performance by utilizing cache allocation technology. *Intel Corporation, April* (2015).

[34] Intel Corporation. 2024. *Intel 64 and IA-32 Architectures Software Developer's Manual*. Intel Corporation. https://cdrdv2.intel.com/v1/dl/getContent/671200

[35] Gorka Irazoqui, Thomas Eisenbarth, and Berk Sunar. 2015. S $ A: A shared cache attack that works across cores and defies VM sandboxing–and its application to AES. In *2015 IEEE Symposium on Security and Privacy*. IEEE, 591–604.

[36] Mehmet Kayaalp, Nael Abu-Ghazaleh, Dmitry Ponomarev, and Aamer Jaleel. 2016. A high-resolution side-channel attack on last-level cache. In *Proceedings of the 53rd Annual Design Automation Conference*. 1–6.

[37] Tom Kessous and Niv Gilboa. 2024. Prune+ PlumTree-finding eviction sets at scale. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 3754–3772.

[38] Jinchun Kim, Elvira Teran, Paul V Gratz, Daniel A Jiménez, Seth H Pugsley, and Chris Wilkerson. 2017. Kill the program counter: Reconstructing program behavior in the processor cache hierarchy. *Acm Sigplan Notices* 52, 4 (2017), 737–749.

[39] Vladimir Kiriansky, Ilia Lebedev, Saman Amarasinghe, Srinivas Devadas, and Joel Emer. 2018. DAWG: A defense against cache timing attacks in speculative execution processors. In *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 974–987.

[40] Paul Kocher, Jann Horn, Anders Fogh, Daniel Genkin, Daniel Gruss, Werner Haas, Mike Hamburg, Moritz Lipp, Stefan Mangard, Thomas Prescher, et al. 2020. Spectre attacks: Exploiting speculative execution. *Commun. ACM* 63, 7 (2020), 93–101.

[41] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. 2015. Deep learning. *nature* 521, 7553 (2015), 436–444.

[42] Huaicheng Li, Daniel S. Berger, Lisa Hsu, Daniel Ernst, Pantea Zardoshti, Stanko Novakovic, Monish Shah, Samir Rajadnya, Scott Lee, Ishwar Agarwal, Mark D. Hill, Marcus Fontoura, and Ricardo Bianchini. 2023. Pond: CXL-Based Memory Pooling Systems for Cloud Platforms. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (Vancouver, BC, Canada) *(ASPLOS 2023)*. Association for Computing Machinery, New York, NY, USA, 574–587. https://doi.org/10.1145/3575693.3578835

[43] Jens Lindemann. 2024. Faking deduplication to prevent timing side-channel attacks on memory deduplication. In *2024 Annual Computer Security Applications Conference (ACSAC)*. IEEE, 380–392.

[44] Fangfei Liu, Qian Ge, Yuval Yarom, Frank Mckeen, Carlos Rozas, Gernot Heiser, and Ruby B Lee. 2016. Catalyst: Defeating last-level cache side channel attacks in cloud computing. In *2016 IEEE international symposium on high performance computer architecture (HPCA)*. IEEE, 406–418.

[45] Fangfei Liu, Yuval Yarom, Qian Ge, Gernot Heiser, and Ruby B Lee. 2015. Last-level cache side-channel attacks are practical. In *2015 IEEE symposium on security and privacy*. IEEE, 605–622.

[46] Jinshu Liu, Hamid Hadian, Yuyue Wang, Daniel S Berger, Marie Nguyen, Xun Jian, Sam H Noh, and Huaicheng Li. 2025. Systematic cxl memory characterization and performance analysis at scale. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 1203–1217.

[47] Artemiy Margaritov, Dmitrii Ustiugov, Edouard Bugnion, and Boris Grot. 2019. Prefetched address translation. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*. 1023–1036.

[48] Microsoft. [n. d.]. Isolation in the Azure Public Cloud. https://learn.microsoft.com/en-us/azure/security/fundamentals/isolation-choices

[49] Fernando Mosquera, Krishna Kavi, Gayatri Mehta, and Lizy K John. 2023. Guard cache: creating false cache hits and misses to mitigate Side-Channel attacks. In *2023 Silicon Valley Cybersecurity Conference (SVCC)*. IEEE, 1–8.

[50] Nasdaq. 2020. IBM Reveals Next-Generation IBM POWER10 Processor. https://www.nasdaq.com/press-release/ibm-reveals-next-generation-ibm-power10-processor-2020-08-17?utm_source=chatgpt.com. Accessed: 2026-04-08.

[51] Divya Ojha and Sandhya Dwarkadas. 2021. TimeCache: using time to eliminate cache side channels when sharing software. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 375–387.

[52] Yossef Oren, Vasileios P Kemerlis, Simha Sethumadhavan, and Angelos D Keromytis. 2015. The spy in the sandbox: Practical cache attacks in javascript and their implications. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*. 1406–1418.

[53] Dag Arne Osvik, Adi Shamir, and Eran Tromer. 2006. Cache attacks and countermeasures: the case of AES. In *Topics in Cryptology–CT-RSA 2006: The Cryptographers' Track at the RSA Conference 2006, San Jose, CA, USA, February 13-17, 2005. Proceedings*. Springer, 1–20.

[54] Chang Hyun Park, Ilias Vougioukas, Andreas Sandberg, and David Black-Schaffer. 2022. Every walk'sa hit: making page walks single-access cache hits. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 128–141.

[55] Sungbo Park, Ingab Kang, Yaebin Moon, Jung Ho Ahn, and G Edward Suh. 2021. Bcd deduplication: Effective memory compression using partial cache-line deduplication. In *Proceedings of the 26th ACM international conference on architectural support for programming languages and operating systems*. 52–64.

[56] Peter Pessl, Daniel Gruss, Clémentine Maurice, Michael Schwarz, and Stefan Mangard. 2016. {DRAMA}: Exploiting {DRAM} addressing for {Cross-CPU} attacks. In *25th USENIX security symposium (USENIX security 16)*. 565–581.

[57] pmem.io. 2019. Using the memmap Kernel Option | Persistent Memory Documentation. https://docs.pmem.io/persistent-memory/getting-started-guide/creating-development-environments/linux-environments/linux-memmap

[58] Meikel Poess and Chris Floyd. 2000. New TPC benchmarks for decision support and web commerce. *SIGMOD Rec.* 29, 4 (Dec. 2000), 64–71. https://doi.org/10.1145/369275.369291

[59] Antoon Purnal, Furkan Turan, and Ingrid Verbauwhede. 2021. Prime+ Scope: Overcoming the observer effect for high-precision cache contention attacks. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. 2906–2920.

[60] Kartik Ramkrishnan, Stephen McCamant, Pen Chung Yew, and Antonia Zhai. 2020. First time miss: Low overhead mitigation for shared memory cache side channels. In *Proceedings of the 49th International Conference on Parallel Processing*. 1–11.

[61] Inc. Red Hat. 2025. What is a Linux container? https://www.redhat.com/en/topics/containers/whats-a-linux-container. Accessed: 2025-Nov.

[62] Red Hat, Inc. 2017. *Red Hat Enterprise Linux 6: Virtualization Administration Guide*. Red Hat, Inc. https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/6/html/virtualization_administration_guide/chap-virtualization_administration_guideadvanced Accessed: 2025-Nov.

[63] Jee Ho Ryoo, Nagendra Gulur, Shuang Song, and Lizy K John. 2017. Rethinking TLB designs in virtualized environments: A very large part-of-memory TLB. *ACM SIGARCH Computer Architecture News* 45, 2 (2017), 469–480.

[64] Gururaj Saileshwar and Moinuddin Qureshi. 2021. {MIRAGE}: Mitigating {Conflict-Based} cache attacks with a practical {Fully-Associative} design. In *30th USENIX Security Symposium (USENIX Security 21)*. 1379–1396.

[65] Gururaj Saileshwar and Moinuddin K Qureshi. 2019. Lookout for zombies: Mitigating flush+ reload attack on shared caches by monitoring invalidated lines. *arXiv preprint arXiv:1906.02362* (2019).

[66] Ali Shafiee, Akhila Gundu, Manjunath Shevgoor, Rajeev Balasubramonian, and Mohit Tiwari. 2015. Avoiding information leakage in the memory controller with fixed service policies. In *Proceedings of the 48th International Symposium on Microarchitecture*. 89–101.

[67] John Peter Stevenson. 2013. *Fine-grain in-memory deduplication for large-scale workloads*. Stanford University.

[68] Synopsys, Inc. [n. d.]. Silicon to Systems Design Solutions. https://www.synopsys.com.

[69] Qinhan Tan, Zhihua Zeng, Kai Bu, and Kui Ren. 2020. PhantomCache: Obfuscating cache conflicts with localized randomization.. In *NDSS*.

[70] Gemma Team, Thomas Mesnard, Cassidy Hardin, Robert Dadashi, Surya Bhupatiraju, Shreya Pathak, Laurent Sifre, Morgane Rivière, Mihir Sanjay Kale, Juliette Love, et al. 2024. Gemma: Open models based on gemini research and technology. *arXiv preprint arXiv:2403.08295* (2024).

[71] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).

[72] Stephan Van Schaik, Cristiano Giuffrida, Herbert Bos, and Kaveh Razavi. 2018. Malicious management unit: Why stopping cache attacks in software is harder than you think. In *27th USENIX Security Symposium (USENIX Security 18)*. 937–954.

[73] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).

[74] Pepe Vila, Boris Köpf, and José F Morales. 2019. Theory and practice of finding eviction sets. In *2019 IEEE Symposium on Security and Privacy (SP)*. IEEE, 39–54.

[75] Xin Wang, Jagadish B Kotra, and Xun Jian. 2022. Eager memory cryptography in caches. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 693–709.

[76] Yao Wang, Andrew Ferraiuolo, and G Edward Suh. 2014. Timing channel protection for a shared memory controller. In *2014 IEEE 20th International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 225–236.

[77] Mario Werner, Thomas Unterluggauer, Lukas Giner, Michael Schwarz, Daniel Gruss, and Stefan Mangard. 2019. {ScatterCache}: thwarting cache attacks via cache set randomization. In *28th USENIX Security Symposium (USENIX Security 19)*. 675–692.

[78] Yuval Yarom and Katrina Falkner. 2014. {FLUSH+ RELOAD}: A high resolution, low noise, l3 cache {Side-Channel} attack. In *23rd USENIX security symposium (USENIX security 14)*. 719–732.

[79] Hosein Yavarzadeh, Archit Agarwal, Max Christman, Christina Garman, Daniel Genkin, Andrew Kwong, Daniel Moghimi, Deian Stefan, Kazem Taram, and Dean Tullsen. 2024. Pathfinder: High-resolution control-flow attacks exploiting the conditional branch predictor. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 770–784.

[80] Yinqian Zhang, Ari Juels, Michael K Reiter, and Thomas Ristenpart. 2014. Cross-tenant side-channel attacks in PaaS clouds. In *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*. 990–1003.

[81] Zirui Neil Zhao, Adam Morrison, Christopher W Fletcher, and Josep Torrellas. 2024. Last-level cache side-channel attacks are feasible in the modern public cloud. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 582–600.

[82] Yongye Zhu, Boru Chen, Zirui Neil Zhao, and Christopher W Fletcher. 2025. Controlled preemption: Amplifying side-channel attacks from userspace. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 162–177.


# A ARTIFACT DESCRIPTION APPENDIX

## A.1 Abstract

Our artifact contains the source code, data, and scripts needed to validate the key results of this paper. We provide: (1) the FPGA bitstream of the CPM, (2) the Linux kernel implementation for using CPM, and (3) the victim and attacker C++ program for security evaluation.

## A.2 Artifact Check-list

## A.3 Description

## A.4 Experiment Workflow

*A.4.1 Flush+Reload Attack. E1: Under VM:* Run attacker and victim in two separate VMs through QEMU:

```
cd qemu
./connect_attacker.sh
./connect_victim.sh
```

Open one terminal to login into victim VM:

```
./ssh_victim.sh
```

Open another terminal to login into attacker VM:

```
./ssh_attacker.sh
```

## A.5 Evaluation and Expected Results

## A.6 Notes and Limitations